Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 11

по курсу «Методы оптимизации»

«Метод Д.Ф.П»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Реализовать метод Д.Ф.П.

2 Результаты

Исходный код программы представлен в листинге 1.

```
using LinearAlgebra
using Plots

function rosenbrock(x)
    n = length(x)
    sum(100 * (x[i+1] - x[i]^2)^2 + (1 - x[i])^2 for i in 1:n-1)
end

function rosenbrock_grad(x)
    n = length(x)
    g = zeros(n)
    for i in 1:n-1
        g[i] += -400 * x[i] * (x[i+1] - x[i]^2) - 2 * (1 - x[i])
        g[i+1] += 200 * (x[i+1] - x[i]^2)
    end
    g
end

function schwefel(x)
    418.9829 * length(x) - sum(x[i] * sin(sqrt(abs(x[i]))) for i in
     eachindex(x))
end

function schwefel_grad(x)
    n = length(x)
    g = zeros(n)
    for i in 1:n
        xi = x[i]
        if abs(xi) > 1e-10
            g[i] = -sin(sqrt(abs(xi))) - xi * cos(sqrt(abs(xi))) *
    (0.5 / sqrt(abs(xi))) * sign(xi)
        end
    end
    g
end
```

```

function rastrigin(x)
    10 * length(x) + sum(x[i]^2 - 10 * cos(2 *    * x[i]) for i in eachindex(x))
end

function rastrigin_grad(x)
    2 * x .+ 20 *    * sin.(2 *    * x)
end

function line_search(f, g, x, d; c1=1e-4, c2=0.9, max_iter=100)
    alpha = 1.0
    phi0 = f(x)
    dphi0 = dot(g(x), d)
    for _ in 1:max_iter
        phi_alpha = f(x + alpha * d)
        if phi_alpha <= phi0 + c1 * alpha * dphi0
            g_new = g(x + alpha * d)
            if dot(g_new, d) >= c2 * dphi0
                return alpha
            end
        end
        alpha *= 0.5
    end
    alpha
end

function dfp(f, grad_f, x0; tol=1e-8, max_iter=1000)
    n = length(x0)
    x = copy(x0)
       = Matrix{Float64}(I, n, n)
    path = [copy(x)]
    g = grad_f(x)
    for k in 1:max_iter
        if norm(g) < tol
            break
        end
        d = -    * g
           = line_search(f, grad_f, x, d)
         x  =     * d
        x = x +  x
        push!(path, copy(x))
        g_new = grad_f(x)
         g  = g_new - g
        g = g_new
```

```
        dot_ x_ g = dot( x ,  g )
        if dot_ x_ g > 1e-14
            z =    *  g
               =    + ( x  *  x ') / dot_ x_ g - (z * z') / dot( g , z)
        end
    end
    n_iter = length(path) - 1
    x, path, n_iter
end

function plot_trajectory_2d(f, x_min, x_max, path, title_str; resolution=200)
    xs = range(x_min[1], x_max[1], length=resolution)
    ys = range(x_min[2], x_max[2], length=resolution)
    Z = [f([x, y]) for x in xs, y in ys]
    p = contour(xs, ys, Z', levels=30, color=:viridis, fill=true)
    plot!(p, [pt[1] for pt in path], [pt[2] for pt in path], color=:red, lw=2, label="")
    scatter!(p, [path[1][1]], [path[1][2]], color=:green, markersize=8, label="          ")
    scatter!(p, [path[end][1]], [path[end][2]], color=:blue, markersize=8, label="          ")
    title!(p, title_str)
    xlabel!(p, " x  ")
    ylabel!(p, " x  ")
    p
end

x0_rosen = [-1.5, 2.0]
x_opt_rosen, path_rosen, iter_rosen = dfp(rosenbrock, rosenbrock_grad, x0_rosen)
p1 = plot_trajectory_2d(rosenbrock, [-2.0, -1.0], [2.5, 3.0], path_rosen, "                    ")
savefig(p1, "rosenbrock_dfp.png")

x0_schwefel = [400.0, 400.0]
x_opt_schwefel, path_schwefel, iter_schwefel = dfp(schwefel, schwefel_grad, x0_schwefel)
p2 = plot_trajectory_2d(schwefel, [250, 250], [450, 450], path_schwefel, "                 ")
savefig(p2, "schwefel_dfp.png")

x0_rastrigin = [1.0, 1.0]
```

```
x_opt_rastrigin, path_rastrigin, iter_rastrigin = dfp(rastrigin, rastrigin_grad, x0_rastrigin)
p3 = plot_trajectory_2d(rastrigin, [-5.0, -5.0], [5.0, 5.0], path_rastrigin, "                    ")
savefig(p3, "rastrigin_dfp.png")

println("                    :  x* = ", x_opt_rosen, "  f(x*) = ", rosenbrock(x_opt_rosen), "                    : ", iter_rosen)
println("                :   x* = ", x_opt_schwefel, "  f(x*) = ", schwefel(x_opt_schwefel), "                    : ", iter_schwefel)
println("                    : x* = ", x_opt_rastrigin, "  f(x*) = ", rastrigin(x_opt_rastrigin), "                    : ", iter_rastrigin)
```

Результат представлен на рисунках 1– 3.

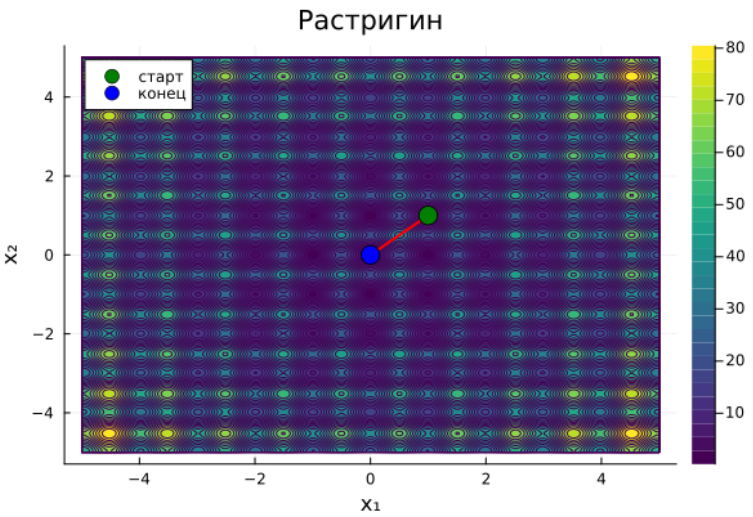

Рис. 1 — Результат

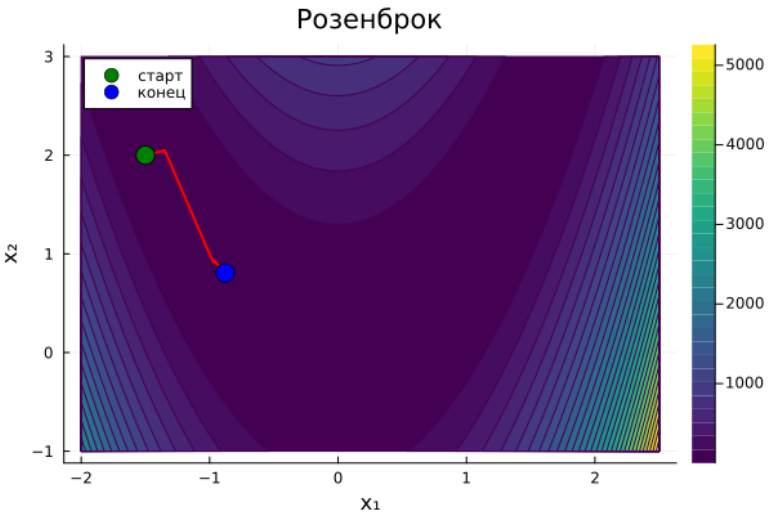

Рис. 2 — Результат

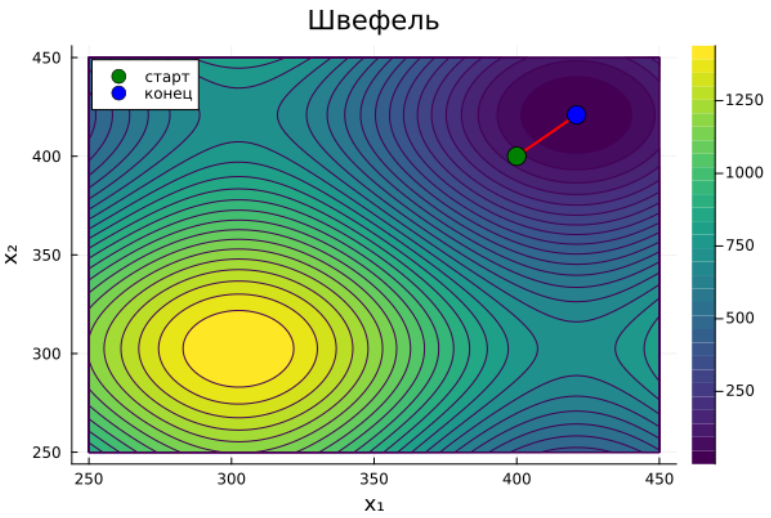

Рис. 3 — Результат